The Struggler — Relatório de entrega 0.3.3

Projeto: BERSERK — The Struggler (mod Forge para Minecraft Java 1.20.1) **Versão entregue:** 0.3.3 **Data:** 11/11/2026 **Branch:** `feature/midland-guard-refinement-0.3.3` (commit `8256acd`, derivada da tag `v0.3.2`) **Tag:** `v0.3.3` **Release:** https://github.com/GustaBackend/The-Struggler/releases/tag/v0.3.3

---

1. Resumo executivo

A versão 0.3.3 não adiciona conteúdo novo ao jogo: ela torna auditável o que já existia e corrige um defeito encontrado em execução.

Foram três mudanças de engenharia e uma correção:

1. A decisão de **quando o escudo bloqueia dano** deixou de ser uma cadeia de condicionais dentro do método `hurt()` e passou a ser uma **política explícita e testável** (`GuardDamagePolicy`), baseada nas tags reais de tipo de dano do Forge.
2. A **escolha da animação** deixou de estar espalhada pelo código e passou a ser uma **regra pura** (`GuardAnimationPriority`), coberta por testes.
3. **Correção de defeito real:** um Guarda com IA desligada mantinha o escudo erguido indefinidamente. A expiração da defesa foi movida do goal de combate para o `tick()` da entidade.
4. Limpeza de código redundante em três classes de combate.

Tudo foi validado em execução real: servidor dedicado Forge 47.4.28, Java 17, GeckoLib 4.7.2, com **duas instâncias de cliente simultâneas**. Nenhum teste foi executado no computador do solicitante.

Resultado: $^{38}\!/_{38}$ testes unitários aprovados, $^{26}\!/_{26}$ GameTests aprovados no servidor real, seis casos de classificação de dano medidos dentro do jogo com os valores esperados, persistência confirmada após reinício completo do servidor e ciclo de entrada/saída de jogador validado.

---

2. Identificação do artefato

Item	Valor
Arquivo	`struggler-0.3.3.jar`
Tamanho	61.895 bytes
SHA-256	`3537265dff330a7e09fb01a8f39b641cd5f700cee340031d8f58070612e5fde3`
Minecraft	1.20.1
Forge	47.4.28
Java	17
Dependência externa obrigatória	GeckoLib 4.7.2 (Forge 1.20.1) — não embutida no JAR
Verificação	JAR baixado de volta da Release do GitHub e hash conferido

O hash foi conferido depois do upload: o arquivo publicado é byte a byte o mesmo que foi testado.

---

3. Ambiente de teste

Componente	Configuração
Servidor	Forge 47.4.28 dedicado, mundo superplano, dificuldade Normal, `online-mode=false`
Clientes	2 instâncias (RefineQA e RefineQB) em displays virtuais `:104` e `:105`
Renderização	software (`LIBGL_ALWAYS_SOFTWARE=1`) sobre Xvfb 1280×720
Java	OpenJDK 17 (`/usr/lib/jvm/java-17-openjdk-amd64`)
Sistema	Ubuntu em sandbox isolado, sem GPU
Mundo de QA	mundo próprio, separado do repositório, com comandos de diagnóstico habilitados apenas nele

Nenhum arquivo do mundo de teste, nenhuma configuração pessoal e nenhuma credencial do solicitante foram usados. A branch `main` não foi alterada em nenhum momento.

---

4. Mudanças técnicas

4.1 Classificação de dano (`GuardDamagePolicy`)

Antes: o método `hurt()` da entidade acumulava a decisão de bloquear, a redução de 40%, a checagem de direção e o agendamento da animação, tudo em sequência de `if`. Isso funcionava, mas não era testável isoladamente e misturava regra de jogo com ciclo de vida da entidade.

Agora:

- `GuardDamageClass` — enumeração do que o escudo enfrenta: corpo a corpo, projétil físico, explosão, fogo, perfurante, não defensável.
- `GuardDamageFacts` — fatos objetivos extraídos do `DamageSource` (tags do tipo de dano + tipo da entidade direta).
- `GuardDamagePolicy` — função pura que transforma fatos em classe de dano. Nenhuma decisão depende de nome de entidade, mensagem de morte ou texto traduzido.

A classificação usa as tags do Forge, não listas de identificadores: `#bypasses_shield`, `#is_explosion`, `#is_fire`, `#is_projectile`. Consequência prática: mods de terceiros que declarem corretamente suas tags passam a ser tratados sem alteração de código.

4.2 Prioridade de animação (`GuardAnimationPriority`)

A ordem dos clipes passou a ser uma função pura, na sequência: morte → golpe em andamento → reação de dano → bloqueio → demonstração de QA → caminhada/corrida → repouso. A morte é absoluta; um golpe não é cancelado por um acerto; a reação de dano dura apenas os 10 ticks de `hurtTime` e devolve o controle ao estado anterior.

4.3 Correção do escudo sem IA

Defeito: a expiração do escudo estava no `tick` do goal de combate. Um Guarda com `NoAI`, ou fora de combate, nunca executava esse `tick` e mantinha o escudo erguido para sempre — inclusive em `status` com `blocking=true` permanente.

Correção: a expiração foi movida para o `tick()` da entidade, que roda independentemente de IA. Cobertura: teste unitário novo e GameTest com entidade sem IA.

Validação em runtime: guarda com `NoAI` e escudo erguido por comando passou de `blocking=true` (`blockTicks=17`) para `blocking=false` (`blockTicks=0`) sozinho, sem intervenção.

4.4 Limpeza

Removidas constantes não utilizadas em `GuardCombatTuning` e `GuardDefenseRules`, e uma condição de alcance redundante em `GuardCombatGoal` (o goal já é acionado dentro do alcance).

4.5 Arquivos tocados

33 arquivos, 975 inserções e 67 remoções. Núcleo: `GuardDamagePolicy`, `GuardDamageFacts`, `GuardDamageClass`, `GuardAnimationPriority`, `MidlandGuard`, `GuardCombatGoal`,

`GuardCombatTuning`, `GuardDefenseRules`, `GuardDamagePolicyGameTests`, `GuardDefenseGameTests`, `GuardDamagePolicyTest`, `GuardAnimationPriorityTest`, além de relatório, evidências e README.

---

5. Testes automatizados

5.1 Unitários (JUnit, sem Minecraft)

Classe	Casos	Falhas
`GuardDamagePolicyTest`	12	0
`GuardAnimationPriorityTest`	10	0
`GuardDefenseRulesTest`	10	0
`GuardCombatTimelineTest`	6	0
Total	**38**	**0**

5.2 GameTests (servidor Minecraft real)

$^{26}/_{26}$ testes obrigatórios aprovados — mensagem literal do servidor de testes: `All 26 required tests passed`.

Cobrem combate (perseguição, golpe único, cooldown, retaliação), defesa (bloqueio frontal, direção, expiração, piso de dano) e classificação de dano (corpo a corpo, projétil, magia, fogo, perfurante). Log transcrito em `docs/midland-guard-refinement/evidence/gametest-033b.log`.

---

6. Testes de execução real

6.1 Classificação de dano com o escudo erguido

Método: vida lida no servidor antes e depois de cada golpe, mais os contadores internos do Guarda (`blockedHits`, `prevented`) expostos por `strugglerguard status`. Redução esperada: 40% no frontal, 0% pelas costas, piso de 1 de dano.

Golpe aplicado	Vida antes → depois	Dano efetivo	`blockedHits`	`prevented`	Esperado	Resultado
Espada frontal (`player_attack` 10)	24 → 18	6	0 → 1	0 → 4.0	6	**APROVADO**
Flecha frontal (`arrow` 10, entidade direta = flecha)	24 → 18	6	1 → 2	4.0 → 8.0	6	**APROVADO**
Magia (`magic` 10)	18 → 8	10	sem alteração	sem alteração	10	**APROVADO**
Fogo (`on_fire` 10)	24 → 14	10	sem alteração	sem alteração	10	**APROVADO**
Bigorna (`falling_anvil` 10, `bypasses_shield`)	24 → 14	10	sem alteração	sem alteração	10	**APROVADO**
Golpe pelas costas (`player_attack` 10)	24 → 14	10	sem alteração	sem alteração	10	**APROVADO**

O `prevented` subindo exatamente 4,0 por acerto bloqueado é a prova de que a redução é aplicada uma única vez e no valor certo — não há dupla aplicação nem desconto acumulado.

Detalhe técnico descoberto em execução: no comando vanilla `/damage <alvo> <valor> <tipo> by <entidade> from <causa>`, o argumento `by` define a **entidade direta** do dano. Para simular uma flecha real é necessário `by <flecha> from <atirador>`; na ordem inversa o jogador passa a ser a entidade direta, o golpe deixa de ser tratado como projétil e não é bloqueado. Isso foi observado duas vezes no teste e está registrado no relatório do repositório.

6.2 Persistência após reinício completo do servidor

Medida	Antes do reinício	Depois do reinício
Posição do Guarda	`[1.5, 100.0, 6.0]`	`[1.5, 100.0, 6.0]`
Vida	`18.0`	`18.0`

Resultado: **APROVADO** — entidade, posição, vida e estado de combate sobrevivem ao save e ao restart.

6.3 Expiração do escudo sem IA

Guarda com `NoAI`, escudo erguido por comando, sem combate algum: o estado voltou a `blocking=false` sozinho, com `blockTicks=0`. Resultado: **APROVADO** — a correção vale em runtime, não só no teste unitário.

6.4 Carregamento

Teste	Resultado
Forge carrega o JAR sem erro de dependência	**APROVADO**
GeckoLib inicializa e o renderer do Guarda registra	**APROVADO**
Ovo de invocação cria `struggler:midland_guard`	**APROVADO**
Modelo, textura e animações carregam sem fallback	**APROVADO**

6.5 Estados e animações no cliente real

Estado	Como foi acionado	Resultado
Idle	guarda parado no mundo	**APROVADO**
Bloqueio	comando de QA e combate real	**APROVADO** — pose distinta, escudo à frente do torso
Ferimento	dano real de 2 com escudo erguido	**APROVADO** — piscar vermelho, recuo e retorno ao bloqueio
Ataque	ação de QA com keyframes do GeckoLib	**APROVADO** — rotação de torso e braço da espada
Caminhada / corrida	deslocamento real, medido por posição	**APROVADO** (mesmo renderer, rodada anterior)
Morte	dano letal	**NÃO EXECUTÁVEL nesta rodada**

6.6 Dois jogadores simultâneos

Teste	Resultado
Dois clientes no mesmo servidor	**APROVADO** — `RefineQA` e `RefineQB` simultâneos
Estado do escudo visível igual nos dois clientes	**APROVADO**
Saída de um jogador	**APROVADO** — `RefineQB left the game`; guarda e mundo continuam
Retorno do jogador	**APROVADO** — `RefineQB joined the game`; o outro jogador segue visível

---

7. Evidências

Dezesseis capturas do cliente real, feitas durante os testes, mais o log transcrito do servidor de GameTests, todas versionadas em `docs/midland-guard-refinement/evidence/`.

Arquivo	Conteúdo
`01-clientA-view.png`	cliente A no mundo de QA, Guarda à frente
`02-clientB-view.png`	cliente B vendo o mesmo cenário
`03-clientA-now.png`	estado do cliente A antes da rodada de dano
`04-clientA-rejoined.png`	cliente A reconectado após expulsão de QA
`05-clientA-respawn.png`	renascimento do jogador A
`06-clientA-after-restart.png`	cliente A após reinício do servidor
`07-clientB-after-restart.png`	cliente B após reinício do servidor
`08-clientA-respawned.png`	jogador A reposicionado no cenário
`09-block-clientA.png`	escudo erguido, visto pelo cliente A
`10-block-clientB.png`	o mesmo escudo erguido, visto pelo cliente B
`11-idle-clientA.png`	Guarda em idle, para comparação
`12-hurt-while-blocking.png`	reação de dano com escudo erguido
`13-back-to-block.png`	retorno ao bloqueio após o ferimento
`14-attack-windup.png`	preparação do golpe
`15-attack-swing.png`	outro instante do golpe
`16-clientB-rejoined.png`	cliente B de volta ao mundo
`gametest-033b.log`	log do servidor de GameTests, $^{26}\!/_{26}$

As capturas 09 e 10 são o par mais importante da rodada: mostram o mesmo estado de escudo, no mesmo instante, em dois clientes independentes. É a prova visual de que o estado é sincronizado pelo servidor e não simulado em cada cliente.

---

8. O que não foi executado e por quê

Animação de morte em 0.3.3 — NÃO EXECUTÁVEL. O Guarda usado na bateria de dano morreu de fato (entidade removida do mundo), e a captura da queda não chegou a ser obtida antes da remoção. O clipe `death`, sua prioridade absoluta e o GameTest correspondente continuam válidos e cobertos pela suíte ($^{26}\!/_{26}$), mas esta rodada não produziu captura nova da morte. Registrado como pendência de evidência, não como falha funcional.

Nada mais da especificação ficou sem execução.

9. Limitações conhecidas

1. **Legibilidade do bloqueio.** A pose é distinta, mas discreta a distância: o escudo sobe cerca de 0,4 bloco e gira a face para frente. Se a leitura visual precisar ser óbvia em combate, o próximo ajuste é ampliar o arco do braço esquerdo no clipe `block`.
2. **Dano de queda** não é defensável por decisão de design (não é golpe direcional) e não foi testado como bloqueável.
3. **Peaceful** ficou fora do escopo por decisão do solicitante.
4. **Desempenho.** Os dois clientes rodaram com renderização por software; os números não representam um cliente com GPU.
5. **Fireball** é classificado como fogo, porque as duas tags do vanilla o marcam assim, e portanto não é bloqueado. Mudar isso é uma linha em `GuardDamagePolicy.classify`.

---

10. Histórico de versões

Versão	Entrega
0.1.0	Fundação Forge, registro do Guarda, ovo, aba criativa, Behelit, diálogo
0.2.0	Visual vanilla com capacete fechado, GeckoLib, sete clipes de animação, arquivo Blockbench editável
0.3.1	Combate autoritativo no servidor, linha do tempo do golpe, GameTests
0.3.2	Defesa com escudo: 40% frontal, 0% pelas costas, piso de 1 de dano
0.3.3	**Política de dano explícita, prioridade de animação, correção do escudo sem IA**

Cada versão tem tag e Release próprias no GitHub, com JAR e evidências. A `main` permanece na fundação; todo o desenvolvimento ocorre em branches por versão.

---

11. Publicação e integridade

- Branch publicada: `feature/midland-guard-refinement-0.3.3`
- Commit: `8256acd` — *refactor: make guard damage blocking explicit and fix shield expiry without AI (0.3.3)*
- Tag anotada: `v0.3.3`
- Release com três anexos: `struggler-0.3.3.jar`, `struggler-0.3.3-entrega.zip` (JAR + relatório + log + 16 capturas) e `relatorio-0.3.3.md`

- Release verificada como publicada (não rascunho), com os três anexos presentes e o JAR baixado de volta com hash idêntico ao testado

---

12. Conclusão

A 0.3.3 entrega o que foi pedido: o comportamento de defesa agora é explícito, testável e verificável no jogo; a escolha de animação virou regra única em vez de lógica dispersa; e o defeito do escudo eterno em entidade sem IA foi corrigido e comprovado em runtime.

O que existe hoje é uma base auditável: 38 testes unitários, 26 GameTests e uma rodada de execução real com dois clientes, dano medido dentro do jogo e evidências versionadas. A pendência aberta é apenas de evidência visual da morte, não de funcionalidade.

Nenhuma funcionalidade nova foi iniciada nesta versão.